

Matrix

A matrix (plural matrices) is a rectangular array of numbers, symbols, or expressions, arranged in rows and columns that is interpreted and manipulated in certain prescribed ways. One such way is to state the order of the matrix. For example, the order of a matrix is represented as 2x3, because there are two rows and three columns. The individual items in a matrix are called its elements or entries.

NOTE

Two matrices can be **added** or **subtracted** element by element **only if they are the same size** (have the same number of rows and the same number of columns).

The rule for **matrix multiplication**, however, is that two matrices can be **multiplied only when the number of columns in the first equals the number of rows in the second**.

Transpose of a matrix is the matrix which is formed by turning all the [rows](#) of a given matrix into [columns](#) and vice-versa. The transpose of matrix A is written A^T .

Example:
$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

A **symmetric matrix** is a [square matrix](#) that is equal to its [transpose](#). Formally, matrix A is symmetric if

$$A = A^T.$$

Because equal matrices have equal dimensions, only square matrices can be symmetric.

The entries of a symmetric matrix are symmetric with respect to the [main diagonal](#). So if the entries are written as $A = (a_{ij})$, then $a_{ij} = a_{ji}$, for all indices i and j .

The following 3×3 matrix is symmetric:

$$\begin{bmatrix} 1 & 7 & 3 \\ 7 & 4 & -5 \\ 3 & -5 & 6 \end{bmatrix}.$$

The various matrix manipulation operations include matrix addition, subtraction, multiplication, finding trace of 2 matrices, finding transpose of a matrix etc.

C Programming Multidimensional Arrays

C programming language allows programmer to create arrays of arrays known as multidimensional arrays. For example:

```
float a[2][6];
```

Here, a is an array of two dimension, which is an example of multidimensional array.

For better understanding of multidimensional arrays, array elements of above example can be considered as below:

	col 1	col 2	col 3	col 4	col 5	col 6
row 1	$a[0][0]$	$a[0][1]$	$a[0][2]$	$a[0][3]$	$a[0][4]$	$a[0][5]$
row 2	$a[1][0]$	$a[1][1]$	$a[1][2]$	$a[1][3]$	$a[1][4]$	$a[1][5]$

Figure: Multidimensional Arrays

Initialization of Multidimensional Arrays

In C, multidimensional arrays can be initialized in different number of ways.

```
int c[2][3]={{1,3,0}, {-1,5,9}};
```

OR

```
int c[][3]={{1,3,0}, {-1,5,9}};
```

OR

```
int c[2][3]={1,3,0,-1,5,9};
```

Initialization of three-dimensional Array

```
double cprogram[3][2][4]={  
{{-0.1, 0.22, 0.3, 4.3}, {2.3, 4.7, -0.9, 2}},  
{{0.9, 3.6, 4.5, 4}, {1.2, 2.4, 0.22, -1}},  
{{8.2, 3.12, 34.2, 0.1}, {2.1, 3.2, 4.3, -2.0}}  
};
```

Suppose there is a multidimensional array $arr[i][j][k][m]$. Then this array can hold $i*j*k*m$ numbers of data.

Matrix Addition

Program:

```
#include <stdio.h>

int main(){
    int r,c,a[100][100],b[100][100],sum[100][100],i,j;
    printf("Enter number of rows (between 1 and 100): ");
    scanf("%d",&r);
    printf("Enter number of columns (between 1 and 100): ");
    scanf("%d",&c);
    printf("\nEnter elements of 1st matrix:\n");
    /* Storing elements of first matrix entered by user. */
    for(i=0;i<r;i++)
        for(j=0;j<c;j++)
        {
            printf("Enter element a%d%d: ",i+1,j+1);
            scanf("%d",&a[i][j]);
        }
    /* Storing elements of second matrix entered by user. */
    printf("Enter elements of 2nd matrix:\n");
    for(i=0;i<r;++i)
        for(j=0;j<c;++j)
        {
            printf("Enter element a%d%d: ",i+1,j+1);
            scanf("%d",&b[i][j]);
        }
}
```

```

/*Adding Two matrices */

for(i=0;i<r;i++)

    for(j=0;j<c;j++)

        sum[i][j]=a[i][j]+b[i][j];

/* Displaying the resultant sum matrix. */

printf("\nSum of two matrix is: \n\n");

for(i=0;i<r;i++)

    for(j=0;j<c;j++)

    {

        printf("%d  ",sum[i][j]);

        if(j==c-1)

            printf("\n\n");

    }

return 0;

}

```

Matrix Multiplication

Program:

```

#include <stdio.h>

int main()

{

    int a[10][10], b[10][10], mult[10][10], r1, c1, r2, c2, i, j, k;

    printf("Enter rows and column for first matrix: ");

    scanf("%d%d", &r1, &c1);

    printf("Enter rows and column for second matrix: ");

    scanf("%d%d",&r2, &c2);

```

```
/* If colum of first matrix in not equal to row of second matrix, asking user to enter the size of matrix again. */
```

```
while (c1!=r2)
{
    printf("Error! column of first matrix not equal to row of second.\n\n");
    printf("Enter rows and column for first matrix: ");
    scanf("%d%d", &r1, &c1);
    printf("Enter rows and column for second matrix: ");
    scanf("%d%d",&r2, &c2);
}
```

```
/* Storing elements of first matrix. */
```

```
printf("\nEnter elements of matrix 1:\n");
for(i=0; i<r1; i++)
for(j=0; j<c1; j++)
{
    printf("Enter elements a%d%d: ",i+1,j+1);
    scanf("%d",&a[i][j]);
}
```

```
/* Storing elements of second matrix. */
```

```
printf("\nEnter elements of matrix 2:\n");
for(i=0; i<r2; i++)
for(j=0; j<c2; j++)
{
    printf("Enter elements b%d%d: ",i+1,j+1);
    scanf("%d",&b[i][j]);
}
```

```

/* Initializing elements of matrix mult to 0.*/

for(i=0; i<r1; i++)

for(j=0; j<c2; j++)

{

    mult[i][j]=0;

}

/* Multiplying matrix a and b and storing in array mult. */

for(i=0; i<r1; i++)

for(j=0; j<c2; j++)

for(k=0; k<c1; k++)

{

    mult[i][j]+=a[i][k]*b[k][j];

}

/* Displaying the multiplication of two matrix. */

printf("\nOutput Matrix:\n");

for(i=0; i<r1; i++)

for(j=0; j<c2; j++)

{

    printf("%d ",mult[i][j]);

    if(j==c2-1)

        printf("\n\n");

}

return 0;

}

```

Transpose of a Matrix

Program:

```
#include <stdio.h>

int main()
{
    int a[10][10], trans[10][10], r, c, i, j;

    printf("Enter rows and column of matrix: ");

    scanf("%d %d", &r, &c);

    /* Storing element of matrix entered by user in array a[[]]. */

    printf("\nEnter elements of matrix:\n");

    for(i=0; i<r; i++)
    for(j=0; j<c; j++)
    {
        printf("Enter elements a%d%d: ",i+1,j+1);

        scanf("%d",&a[i][j]);
    }

    /* Displaying the matrix a[[]] */

    printf("\nEnter Matrix: \n");

    for(i=0; i<r; i++)
    for(j=0; j<c; j++)
    {
        printf("%d ",a[i][j]);

        if(j==c-1)

            printf("\n\n");
    }
}
```

```

/* Finding transpose of matrix a[][] and storing it in array trans[][]. */

for(i=0; i<r; i++)
for(j=0; j<c; j++)
{
    trans[j][i]=a[i][j];
}

/* Displaying the transpose,i.e, Displaying array trans[][]. */

printf("\nTranspose of Matrix:\n");

for(i=0; i<c; i++)
for(j=0; j<r; j++)
{
    printf("%d ",trans[i][j]);

    if(j==r-1)
        printf("\n\n");
}

return 0;
}

```